

Determinized Monte Carlo Tree Search for Jaipur: Algorithm, Failure Analysis, and Compute Scaling

Jaipur AI Project
Technical report, 23 September 2026

Abstract

This report documents the current local Jaipur game AI and evaluates whether increasing the number of hidden-state search trees improves play. The default hard agent combines public-action card memory, constrained sampling of unseen cards and bonus tokens, eight independent determinized Monte Carlo search trees, a self-play-trained linear prior, and terminal win/loss backups. We distinguish this deployed configuration from a one-ply heuristic and an experimental neural prior. Paired, seat-swapped native matches show that the current search beats a pinned older implementation 20–0 at 200 ms per move and 6–0 at 1 s per move; these small, correlated samples do not establish a universal win probability. Holding the budget at 200 ms, 16 trees tie eight trees 10–10 over 20 games. Doubling both trees and time to 16 trees/400 ms yields 9–11 against eight trees/200 ms. Thus the available complete-match evidence does not justify increasing the default tree count. Case studies expose sparse sampling of rare opponent hands and bias in the learned continuation policy as more consequential limitations.

1 Scope and research question

Jaipur is a two-player, hidden-hand trading game. A move may collect one market good, collect all market camels, sell a set of one good, or exchange at least two market goods for goods and/or camels. The local AI plays without network services or access to its opponent’s hidden hand. The practical question motivating this report is whether more determinized trees, accompanied by a larger time budget, produce a stronger opponent. We evaluate complete matches rather than only static state-prediction error or isolated move examples.

This document describes the repository working-tree implementation as inspected on 23 September 2026, including uncommitted changes. The measured contests used the native C++ search binary and the TypeScript rules engine; the browser deploys a WebAssembly build of the same C++ decision logic. A native result is evidence about the algorithm under the stated native budget, not a measured browser win rate. The ICLR 2027 style is used for presentation only; this is an internal technical report, not a conference submission.

2 Game model and information boundary

2.1 Rules and objective

The deck contains 55 cards: 6 each of diamond, gold, and silver; 8 each of cloth and spice; 10 leather; and 11 camels. Three camels start in the five-card market. Goods hands are limited to seven cards. Diamond, gold, and silver sales require at least two matching cards; other goods may be sold singly. A sale receives the remaining goods tokens at the front of that good’s public stack, up to the number sold. A sale of at least three cards may also receive one hidden bonus token from the corresponding three-, four-, or five-plus-card stack.

A round ends immediately when three goods-token stacks are empty, or after a market-refilling move leaves fewer than five market cards. Final round score is goods-token value

plus bonus-token value plus five points for the sole camel-majority holder. Ties are broken first by bonus-token count and then by goods-token count. Winning a round earns one seal; the first player to two seals wins the match. The hard AI’s tree value optimizes the current round’s win/draw/loss, not the probability of winning the multi-round match. The authoritative front-end rules are in `src/game/engine.ts`; C++ search transitions mirror them.

2.2 Observation and public card memory

The AI observation contains its own hand and camels; both players’ public goods-token totals and counts; its own known bonus values; the market, public discarded goods, remaining goods-token stacks, counts of remaining bonus tokens, deck size, opponent goods-hand size, and turn number. The game seed, deck order, opponent’s actual hand, and opponent’s secret bonus values are absent from the encoded WebAssembly input.

The public-event replay maintains a lower bound K_g on the number of opponent cards of good g known with certainty. Taking a market card adds it; publicly selling a card removes it; exchanging removes any known cards paid and adds goods taken from the market. The memory resets when a new round is dealt. Initially dealt opponent goods remain unknown. Cards of the same type are indistinguishable, so K_g is a count, not an identity or a probability. This logic resides in `src/game/ai.ts`.

2.3 Hidden-state sampling

For good g , let C_g be the deck’s fixed inventory, H_g the agent’s hand count, M_g the market count, and D_g the publicly discarded count. The residual unseen inventory is

$$U_g = C_g - H_g - M_g - D_g - K_g. \quad (1)$$

The sampler shuffles these residual goods, fills the unknown part of the opponent hand, and assigns the rest to the hidden deck. Camel counts follow from card conservation and the known hand, market, and deck sizes. It also removes the agent’s known bonus values from each bonus pool, then samples the values of the opponent’s already claimed bonuses and the order of future bonuses. Each search tree receives one such complete sampled world. The sampler conditions on public card counts, but does not fit a behavioral model of the opponent’s preference for particular goods.

3 Search algorithm

3.1 Legal root and interior actions

The C++ action generator enumerates one take action per available goods type; selling every permitted count of each held type; collecting all market camels; and each legal exchange specified by goods-type counts for both sides of the exchange. Exchanges require at least two market goods, equal numbers taken and paid, no type simultaneously taken and paid, sufficient camels, and a post-exchange hand of at most seven. Equivalent permutations of identical cards share one action. A tree edge corresponds to one complete legal turn. The exchange branching factor can nevertheless be large when the market and camel herd are rich.

3.2 Linear value prior and rollout policy

The default state evaluator uses 70 antisymmetric base features and another 70 features multiplied by game progress $1 - \min(40, d)/40$, where d is the remaining deck size. Features encode current public proceeds, player to move, per-good hand counts, currently sellable goods-token cash, expected sale-bonus tier, camel count, claimed token counts, and camel majority. A weighted sum passed through $\tanh$ gives an approximate value in $[-1, 1]$. The weights were trained by 100,000 self-play rounds using $\text{TD}(\lambda)$, $\lambda = 0.7$, a norm-adjusted step size based on 0.04, and a terminal target of point margin divided by 224. Training used

15% random moves and averaged weights from the latter half of training. Runtime search does not update the weights.

For each legal action at a newly expanded node, the search evaluates the resulting state from the moving player’s perspective. Let v_a be that one-step estimate. The prior is

$$P(a) = \frac{\exp[4(v_a - v_{\max})/\max(0.05, v_{\max} - v_{\min})]}{\sum_b \exp[4(v_b - v_{\max})/\max(0.05, v_{\max} - v_{\min})]}. \quad (2)$$

Every legal action retains positive prior probability. In a rollout, 90% of moves greedily select the legal one-step successor with the highest linear value for the mover; 10% use a random category/action generator. This policy controls the simulated continuation, but its estimate is never substituted for a finished game’s actual outcome.

3.3 Tree selection, backup, and root aggregation

The default hard mode builds eight independent trees under an approximately 1,000 ms total wall-clock budget, nominally one eighth per tree. A fixed-iteration API exists for tests; the deployed timed API caps each tree at 100,000 iterations but normally stops by time. The clock is checked every 16 iterations, so the budget is approximate. The exploration coefficient is $c = \sqrt{2}$. For edge a with prior P_a , visits n_a , parent visits N , and mean root-player outcome Q_a , selection uses

$$S(a) = sQ_a + cP_a \frac{\sqrt{N+1}}{1+n_a}, \quad s = \begin{cases} +1, & \text{root player to act,} \\ -1, & \text{opponent to act.} \end{cases} \quad (3)$$

Expansion stops at a previously unvisited edge. A continuation runs until the round ends or 256 rollout moves have elapsed; tree traversal has a separate 128-edge guard. Terminal returns are +1, 0, −1 for win, draw, loss from the root player’s perspective. An overlong rollout backs up neutral zero and increments a truncation counter. Exact terminal results, not the linear value, are accumulated on path edges.

Each tree’s root-action visits are divided by that tree’s actual total root visits, giving each sampled world equal aggregate weight. The selected action usually maximizes summed normalized visit share; ties use average outcome. A special fallback uses mean bounded point margin $\tanh((p_0 - p_1)/20)$ only if every aggregated root action was visited and all its sampled outcomes lost. An unvisited candidate blocks this fallback. This is not a general risk-sensitive or imminent-loss override.

The experimental ?ai=neural path replaces node priors with a 160-input, 64-hidden-unit symmetric network. The browser path retains the default linear rollout policy. It is separate from a research profile that replaced both priors and rollouts, which lost 5–35 against the linear search at a nominal 50 ms budget. A lower held-out value-prediction error for the network does not establish better play.

4 Experimental protocol

4.1 Complete-match evaluation

The native benchmark launches the candidate and baseline as separate C++ processes and applies their returned actions to the same TypeScript rules engine used by the browser. For each seed, it plays two full matches with candidate and baseline seats swapped. A match continues through rounds until a player receives two seals. Both agents receive the same nominal budget within a comparison; unequal-budget rows explicitly show both values. The benchmark records winner, seed, seat, final round scores, measured per-move search time, and the complete public action sequence. Seeds, budgets, and outcomes below are fully specified; the generated JSONL logs accompany this report.

The pinned old baseline is commit c86381e. Its research wrapper adds a wall-clock stop, uses actual root visits in vote normalization, and exposes a persistent process interface. Thus it is a reproducible historical search implementation comparator, not a byte-for-byte copy of

a user’s earlier browser build. The older 50 ms result used a different seed set from the 200 ms and 1 s follow-ups. Match pairs share a seed and are correlated, so treating every win as an independent Bernoulli observation would overstate certainty. No confidence interval or claim of optimality is made from these small groups.

4.2 Results against the pinned old search

Table 1: Current eight-tree guided search versus pinned old search. Each pair uses both seat assignments; nominal budgets are per move for both sides.

Seeds	Budget	Games	Current–old	Current mean ms	Old mean ms
1201–1210	50 ms	20	20–0	–	–
2601–2610	200 ms	20	20–0	198.9	201.4
2701–2703	1,000 ms	6	6–0	951.1	994.1

The 50 ms result alone was an inadequate basis for predicting performance at deployment budget. The 200 ms and 1 s follow-ups support the narrower statement that the current whole search implementation beat this specific pinned comparator in these tested native matches. The 1 s group contains only three seed pairs. These results cannot isolate the effect of the recent all-loss point-margin tie-break, nor do they measure performance against a human or an independently strong agent.

4.3 Tree-count and time ablation

Table 2: Tree-count tests against the default eight-tree linear search. Winner counts refer to the first and second configurations, respectively.

Candidate	Comparator	Seeds	Games	Wins
16 trees, 200 ms	8 trees, 200 ms	2801–2810	20	10–10
16 trees, 400 ms	8 trees, 200 ms	2901–2910	20	9–11

In the equal-time experiment, mean measured search times were 204.6 ms for 16 trees and 201.2 ms for eight trees. In the scaled-time experiment they were 399.6 ms and 200.2 ms. Doubling tree count at fixed time trades per-world depth for more sampled hidden worlds; the observed 10–10 provides no evidence of a gain. Doubling both trees and time approximately preserves per-tree budget, yet the observed 9–11 again fails to show a gain. A 16-tree/2 s versus eight-tree/1 s contest was stopped at the user’s request after only three of four planned games (2–1 for 16 trees). That incomplete group is excluded from Table 2 and from the decision. A time-only eight-tree/400 ms control was not run. Consequently these data cannot estimate the separate causal contribution of doubling time with high precision, but they do not justify shipping 16 trees.

5 Failure analysis

Two full-match decision cases show why more root samples or a better continuation policy might matter more than simply increasing iterations. In saved case 1004, the agent held two leather and five camels; public actions proved that the opponent held at least two diamonds. It exchanged four camels for goods, after which the opponent sold three diamonds and won the round 67–62. Selling the two leather instead, followed by the recorded same opponent move, would have won 64–62. In case 1308, the agent held four leather while at least three opponent gold cards were publicly known. It exchanged two camels for two more leather; the opponent then sold five gold, winning 77–69. Selling the four leather before that same recorded reply would have won 78–77.

These are valid one-step counterfactuals under the realized hidden hand and the fixed subsequent reply, not proofs that the alternative maximizes ex-ante win probability. Under the current hidden-card sampler, the exact fatal opponent combinations appeared in approximately 18.4% and 10.7% of 100,000 independent world samples for the two cases. Only one of the deployed eight root worlds contained the fatal combination in each position. The root-vote rule can therefore dilute a serious but sparse threat, particularly if the learned rollout policy overvalues the subsequent exchange. The complete case saves and diagnostic procedure are in the companion evaluation report.

Prior ablations reinforce this caution. Directly reranking promising actions by additional simulation on the same sampled worlds repaired an isolated move but lost 9–31 to the guided baseline over 40 native matches at nominal 50 ms. Raising exploration coefficient to 2.5 scored 19–21. Using neural prior and neural rollout scored 5–35. These low-budget research results are diagnostic rather than deployment-scale rankings; they show that an apparently sensible local fix or lower offline prediction error may fail in full games.

6 Discussion and recommendation

The evidence supports retaining eight trees at the current approximately 1 s browser budget. The 200 ms tree-count ablation is a tie, and the proportional 400 ms test loses narrowly despite double computation. The incomplete 2 s contest is too small to override those complete results. Increasing tree count is a plausible way to cover more rare hidden hands, but it does not address inaccurate continuation play, imperfect opponent modeling, or the root rule’s emphasis on visit share. The all-loss fallback is especially narrow and cannot rescue a move that is actually winning but seldom sampled.

A stronger next iteration should separate two failure sources. First, evaluate alternative legal moves on paired hidden worlds with several continuation policies, so action ordering is less sensitive to one learned greedy rollout. Second, allocate hidden-world samples adaptively to public endgame threats and measure whether the added samples change action ranking. Any change should be tested in seat-swapped, complete native matches at 200 ms and 1 s, followed by browser WebAssembly checks at the actual 1 s budget. Report sampled seeds, elapsed time, terminal and truncated rollout counts, and saved action logs; do not infer strength from a single saved game or a value-network loss alone.

7 Conclusion

The current determinized, learned-policy MCTS is substantially stronger than a pinned older search in the tested native games at both 200 ms and 1 s per move. That finding concerns the combined algorithm, not an individual recent patch. The completed tree-count experiments do not show that 16 trees improve match results, even when their total time budget is doubled at 200 ms scale. Public-action memory and card-conservation sampling are useful, but eight sampled hidden worlds and a single greedy learned continuation policy leave observable endgame failures. The default configuration therefore remains eight trees and approximately 1 s per move while more targeted search and rollout changes are investigated.

Reproducibility and source artifacts

This report uses the working-tree code as of 23 September 2026. Core search: `cpp/jaipur_ai.cpp`, `cpp/mcts.hpp`, `cpp/rollout-policy.hpp`, and `cpp/rollout-weights.hpp`. Observation, rules, and Wasm bridge: `src/game/ai.ts`, `src/game/engine.ts`, `src/game/ai-wasm.ts`, and `src/game/ai.worker.ts`. Native driver: `scripts/benchmark-native.ts` and `scripts/ai-research.cpp`. The report’s data/ directory holds the 200 ms and 1 s complete-match JSONL event logs; the earlier 50 ms logs and case saves are under analysis/ai-eval-2026-09-23/. The `?ai=neural` browser configuration is experimental and is not the default hard mode.